

Solution Design

CURRENTLY IMPLEMENTED | Evidence Before Authority

Business Problem

An ordinary-looking agent upgrade can silently gain order placement, write access or transaction authority while removing human approval. AgentGate compares releases and requires policy and assurance evidence before authority is approved.

Intended Users

Engineering reviewers and accountable release owners assessing a baseline-to-candidate agent change. A local reviewer interface and CLI expose decisions, evidence and required remediation.

Tools and Grounding Data

Controlled JSON manifests, policies, risk taxonomy, test catalogue and stored assurance evidence ground the workflow. Python computes the exact deterministic manifest diff and owns authoritative decision fields.

Why Multi Agent

Three specialized responsibilities separate change explanation, assurance planning and release judgment. This makes handoffs inspectable; strict Python validation, rather than agent agreement, establishes decision authority.

UNSAFE AUTHORITY EXPANSION: BLOCK

Two hard policy blockers stop approval even when ten selected stored results pass. Synthetic demonstration only: no trading or measured commercial impact is claimed.

Agent Architecture

CURRENTLY IMPLEMENTED prototype and clearly separated future services

Specialised Agents

Three Foundry-backed model stages; project-scoped Responses calls, not persistent portal-agent resources. Teal marks model reasoning.

End to End Workflow

IMPLEMENTED PROTOTYPE BOUNDARY

Reviewer input

Fixed scenario, baseline, candidate and execution mode



Python validation + exact manifest diff

Strict schemas and confirmed field-level changes



1 Change Analyst

Explains only the supplied deterministic diff



2 Assurance Planner

Explains controlled risks, policy and selected assurance



Deterministic policy + assurance engine

Checks blockers, stored evidence and the verdict floor



3 Release Judge

Structured recommendation must match the Python floor



Non-downgrade verdict guard

Validate references and reject conflicting model verdicts



Auditable decision

APPROVE / CONDITIONAL / BLOCK + remediation + trace

Python validates every handoff. Responsibility flow, not an exhaustive call graph.

PLANNED PRODUCTION SERVICES - NOT IMPLEMENTED

Authenticated service + enterprise identity/RBAC; managed Foundry agents if appropriate; OpenTelemetry/Foundry trace export to Application Insights; formal evaluators; controlled CI/CD approval; audit and retention services.

Governance and Reliability

CURRENTLY IMPLEMENTED | Model explanations remain advisory

01 Exact comparison

Python validates manifests and computes the exact baseline-to-candidate diff. Model prose cannot add an authority change.

02 Controlled evidence

A bounded taxonomy, policy and test catalogue determine selected tests. Stored results are validated, not freshly executed candidate-agent tests.

03 Structured handoffs

Strict Pydantic contracts and reference checks constrain changes, plans and decisions. Each stage permits at most one structured-output repair.

04 Deterministic blockers

Python owns blockers, conditions, test selection and remediation. The model verdict must equal the deterministic floor in this implementation.

05 Safe failure

Unavailable Foundry calls, invalid output or invalid references stop the cloud review. Local mode is an explicit choice, never a silent replacement.

06 Human accountability

The reviewer inspects the evidence and required remediation. AgentGate issues an assurance decision; it does not deploy a release or place trades.

Model output cannot downgrade validated policy blockers.

Prototype Evidence

CURRENTLY IMPLEMENTED | Three live cases; nine first-attempt stage calls

UNSAFE

BLOCK

11.57 s model calls

SAFE

APPROVE

8.32 s model calls

INCOMPLETE

CONDITIONAL

7.98 s model calls

Live runs: 25 September 2026, 04:55 UTC. Durations sum model requests; they are not browser latency benchmarks. All three stages ran for each case.

Unsafe authority changes

Order-placement tool added | Brokerage READ to WRITE

Transaction authority: USD 0 to USD 5,000

Mandatory human approval removed

Evaluation Strategy

53

automated tests passed

13 / 13

repository evaluation cases passed

13/13 repository cases = 10 labelled cases + 3 demonstrations. Checks cover verdicts and expected control/test subsets. Six BLOCK labels reuse the unsafe manifest; this is not general coverage or formal Foundry evaluation.

Evidence: docs/evidence/live-run-summary.json and scenario trace/decision pairs. Unsafe is blocked; safe is approved; missing evidence is conditional.

Observability Strategy

CURRENTLY IMPLEMENTED versus PRODUCTION-READINESS PLAN

CURRENTLY IMPLEMENTED

Structured local application traces contain stage status, Foundry response IDs, timing, token usage and the final decision. The reviewer downloads decisions, evidence and trace JSON. Trace IDs connect the displayed review to its saved record.

PRODUCTION-READINESS PLAN - NOT IMPLEMENTED

Export OpenTelemetry/Foundry traces to Application Insights. Monitor latency, token use, validation repairs, failure rates, tool success, verdict distribution and policy-blocker frequency. Define alert thresholds and ownership before deployment.

Limitations

Current traces are local JSON, not Azure Monitor or Application Insights. There are no persistent managed agents or formal Foundry evaluation jobs. Tool success is a planned operational metric; this prototype consumes stored test results and does not execute candidate-agent tools.

Model explanations remain advisory and can contain unsupported claims: saved safe-case prose mentions tests Python did not select. Strict schemas and resolvable references do not prove narrative truth. Python-owned decision fields remain authoritative.

Fixed policies and repeated synthetic manifests limit evaluation coverage. The reviewer is a local demonstration without hosted identity, enterprise audit retention or production approval infrastructure. No real trading or commercial impact metrics are claimed.

Production-Readiness Plan

PLANNED ONLY | No production services have been provisioned or verified

Deployment and Continuous Improvement

Deploy the reviewer behind an authenticated service endpoint. Add enterprise identity, role-based access, evidence retention, audit logging and human release approval. Migrate reasoning stages to persistent managed Foundry agents only if operationally appropriate; preserve deterministic non-downgrade controls.

Evaluation Strategy - planned release gate

Use formal Foundry evaluators for groundedness, task adherence, tool usage and response quality. Add representative regression cases whenever policies, tools, models or prompts change. Run evaluations before release and integrate AgentGate into a controlled CI/CD approval step with accountable human sign-off.

Improvement loop: review traces and failures, add regression cases, revise the controlled policy or implementation, evaluate, then seek approval. Model-quality scores must never override deterministic blockers.

Reproduce the implemented prototype

```
git clone https://github.com/Gabriel-vr1/AgentGate.git
cd AgentGate
py -3.12 -m venv .venv
.venv/Scripts/python.exe -m pip install -e '.[dev,foundry]'
.venv/Scripts/python.exe app.py serve
```

Open <http://127.0.0.1:8765>. For live mode, use Azure CLI sign-in and an existing authorized project endpoint and model deployment configured as described in README. Explicit local deterministic mode remains available.

<https://github.com/Gabriel-vr1/AgentGate>

[Evidence index, response IDs, timings and qualifications](#)

Planned services are roadmap items, not prototype capabilities or verified integrations.